CAMP ORDER APP

DATABASE ARCHITECTURE & ENTITY–RELATIONSHIP DIAGRAM

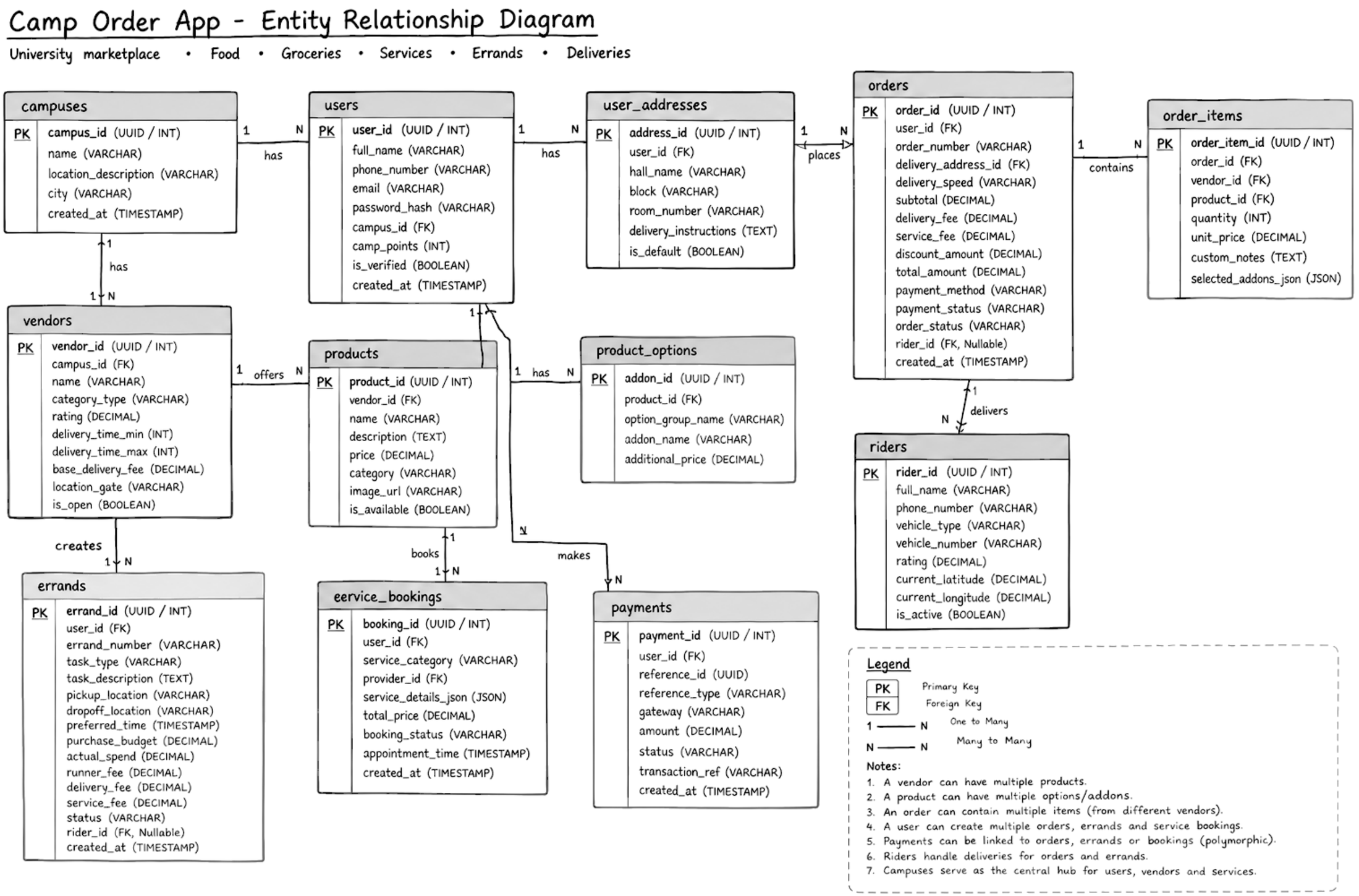
Core database entities, relationships, key fields, and implementation notes

Document purpose	Database design reference for the Camp Order marketplace.
Core entities	Campuses, users, addresses, vendors, products, orders, errands, service bookings, riders, and payments.
Formatting	Black text, white background, simple tables, and print-friendly linework.
Design note	The ERD is a conceptual model. Validate constraints and workflow rules before building migrations.

Prepared for technical planning and implementation.

1. Entity–Relationship Diagram

Overview of the 12 core tables and their principal relationships. Use the table catalogue and notes in the following pages for implementation details.



2. Core Table Catalogue

PK = primary key; FK = foreign key. Suggested data types are illustrative and can be adapted to PostgreSQL or MySQL.

TABLE	KEY FIELDS	ROLE / NOTES
campuses	campus_id PK; name; location_description; city; created_at	A campus or regional hub. Parent for users and vendors.
users	user_id PK; full_name; phone_number UNIQUE; email; password_hash; campus_id FK; camp_points; is_verified; created_at	Customer profile, authentication details, and loyalty balance.
user_addresses	address_id PK; user_id FK; hall_name; block; room_number; delivery_instructions; is_default	Saved delivery locations. An order uses one selected address.
vendors	vendor_id PK; campus_id FK; name; category_type; rating; delivery_time_min; delivery_time_max; base_delivery_fee; location_gate; is_open	Restaurants, shops, pharmacies, and service providers.
products	product_id PK; vendor_id FK; name; description; price; category; image_url; is_available	Items sold by a vendor.
product_options / product_addons	addon_id PK; product_id FK; option_group_name; addon_name; additional_price	Custom options such as portion sizes or extras. Consider separate option-group and option tables as complexity grows.
orders	order_id PK; user_id FK; order_number UNIQUE; delivery_address_id FK; delivery_speed; subtotal; delivery_fee; service_fee; discount_amount; total_amount; payment_method; payment_status; order_status; rider_id FK NULL; created_at	Customer checkout and overall order totals/status.
order_items	order_item_id PK; order_id FK; vendor_id FK; product_id FK; quantity; unit_price; custom_notes; selected_addons_json	Line items within an order, including price and option snapshots.
errands	errand_id PK; user_id FK; errand_number UNIQUE; task_type; task_description; pickup_location; dropoff_location; preferred_time; purchase_budget; actual_spend; runner_fee; delivery_fee; service_fee; status; rider_id FK NULL; created_at	Custom shopping, pickup, and peer-to-peer runner requests.
service_bookings	booking_id PK; user_id FK; service_category; provider_id FK to vendors; service_details_json; total_price; booking_status; appointment_time; created_at	Appointments for services such as printing, laundry, cleaning, or repairs.
riders	rider_id PK; full_name; phone_number UNIQUE; vehicle_type; vehicle_number; rating; current_latitude; current_longitude; is_active	Rider and runner profiles. Store location history separately if live tracking is needed.
payments	payment_id PK; user_id FK; reference_id; reference_type; gateway; amount; status; transaction_ref; created_at	Payment attempts and outcomes for orders, errands, and bookings.

3. Relationship Rules

RELATIONSHIP	RULE
campuses → users	One campus can have many users; each user belongs to a campus (campus_id FK).
campuses → vendors	One campus can have many vendors.
users → user_addresses	One user can save many delivery addresses.
vendors → products	One vendor can list many products.
products → product_options / add-ons	One product can offer multiple configurable options or extras.
users → orders	One user can create many orders.
orders → order_items	One order contains one or more line items.
products / vendors → order_items	Each line item references the product and vendor sold.
users → errands	One user can submit many errands.
users → service_bookings	One user can make many service bookings.
vendors → service_bookings	One vendor/provider can fulfil many bookings.
riders → orders / errands	A rider may be assigned to multiple jobs over time; assignment can be null before dispatch.
users → payments	One user can have multiple payment records.
payments → business records	The reference_type/reference_id design is polymorphic and cannot use a standard FK to all three target tables.

4. Recommended Supporting Tables

TABLE	REASON TO ADD
vendor_orders	Split a multi-vendor checkout into one fulfilment record per vendor; supports independent preparation and dispatch.
order_status_history	Record each status change, timestamp, and actor for support and audit purposes.
delivery_location_updates	Store timestamped rider coordinates rather than overwriting only the rider's current location.
camp_points_ledger	Audit points earned, redeemed, adjusted, and expired. Keep users.camp_points as a cached balance if useful.
payment_events / refunds	Handle gateway callbacks, duplicate-event prevention, reconciliation, and refunds.
product_option_groups	Represent option groups and rules such as required choices and minimum/maximum selections.
notifications	Track order, errand, booking, and delivery notifications.

5. Implementation Notes

Multi-vendor checkout

Because an order can contain products from different vendors, an order-level status and rider may not be sufficient. Use a parent orders row plus vendor_orders rows for each vendor's fulfilment, if mixed-vendor baskets are supported.

Payment integrity

The polymorphic reference in payments is flexible but cannot enforce a conventional FK. Consider a shared checkout/transaction entity or separate nullable FKs with a check constraint.

Money fields

Use an exact decimal type such as NUMERIC(12,2), and store the currency (for example GHS). Do not use floating-point values for money.

Password security

Store only a strong password hash using Argon2id or bcrypt. Never store plain-text passwords.

Payment verification

Mark a payment as successful only after validating a trusted server-to-server gateway callback or verifying the transaction with the provider.

Indexes

Index foreign keys and common queries: products(vendor_id, is_available), vendors(campus_id, is_open), orders(user_id, created_at), orders(order_status), errands(status), bookings(provider_id, appointment_time), and payments(reference_type, reference_id).

Data integrity

Use unique constraints for phone numbers and order/errand numbers; constrain status values; enforce at most one default address per user.

Live tracking

Rider coordinates change frequently. Store current coordinates on riders for quick display and a separate location-update history for route playback.

JSON fields

Use JSON for flexible booking details and selected add-ons, but keep important filter/search fields in typed columns.

Summary

This document combines the ER diagram with a typed table catalogue and implementation guidance. It is intended to be readable on screen and printable in black and white. The diagram is a conceptual overview; confirm the exact schema, constraints, and workflow decisions before creating production migrations.